CUET PG

DAILY CLASS NOTES

Computer Science

Operating System

Lecture – 07

Multithreading



Multithreading

Topics to be Covered

1. Threads
2. Multi-Level Queue
3. Multi-Level Feedback Queue

1. Threads

- The **basic unit of CPU utilization** is called a **thread**.
- A thread includes:
  - **Program Counter (PC):** Indicates the next instruction to execute.
  - **Thread ID:** Unique identifier.
  - **Stack:** Stores local variables, return addresses, and function calls.
  - **Register Set:** Stores processor register values.
- Threads **share** certain resources with other threads belonging to the same process, such as:
  - **Data Section**
  - **Code Section**
  - **Operating System resources** (open files, signals, etc.)

Traditional (Heavyweight) Process

- Has **only one thread of control**.
- If that thread blocks (e.g., waiting for I/O), the **entire process is blocked**.

Motivation for Multithreading

- Most **modern applications** are implemented as **multi-threaded programs**.
- They run as **a single process** containing **multiple threads of control**.
- Common in **desktop applications, web servers, and modern OS design**.

Benefits of Multithreaded Programming

The benefits of multithreading can be divided into **four major categories**:

1. **Responsiveness**
   - Allows parts of a program to continue running even if others are blocked.
   - **Example:** In a web browser, one thread can render text while another loads images.
2. **Resource Sharing**
   - Threads share code and data, making communication between them easy and efficient.
3. **Economy**
   - Creating and managing threads is **faster and cheaper** than creating full processes.
4. **Scalability**
   - On multicore systems, threads can run **in parallel** across different cores, improving performance.

Multicore Programming

- A **multicore system** has **multiple CPU cores on a single chip**.
- Each core appears as a **separate processor** to the operating system.
- **Multithreaded programming** helps use multiple cores effectively, improving **concurrency and throughput**.



Challenges in Multicore Programming

1. **Dividing Activity:**
   - Identify which parts of the program can be split into **independent tasks** that can run in parallel.
2. **Balance:**
   - Ensure that all cores have **equal amounts of work** so that no core remains idle.
3. **Data Splitting:**
   - Separate data so that each thread/core can access and manipulate its required data independently.
4. **Data Dependency:**
   - Identify data dependencies (shared data) between tasks to avoid race conditions and synchronization issues.
5. **Testing and Debugging:**
   - Testing parallel programs is **difficult** because multiple execution paths exist.
   - Debugging multithreaded applications is **harder** than debugging single-threaded ones.

Multithreading Models

Operating systems use different models to **map user-level threads to kernel-level threads.**

1. **Many-to-One Model**
   - Many **user threads** are mapped to **one kernel thread**.
   - Thread management is done in **user space** using a **thread library**, making it efficient.
   - However, if one thread makes a **blocking system call**, the **entire process blocks**.
   - **Example:**

     Java Green Threads
2. **One-to-One Model**
   - Each **user-level thread** maps to a **separate kernel thread**.
   - Allows **true concurrency** — multiple threads can run in parallel on multiprocessor systems.
   - **Drawback:**
     - Creating a user thread requires creating a **kernel thread**, which increases **overhead**.
   - **Example:**

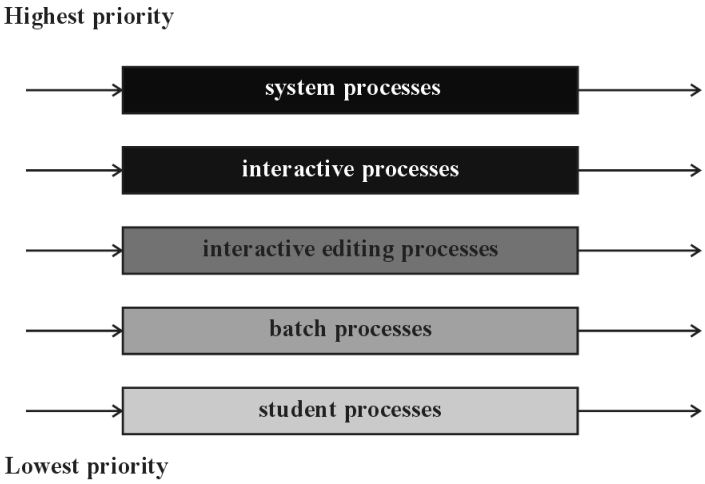
     Windows and Linux systems use this model.
3. **Many-to-Many Model**
   - Maps **many user threads** to an **equal or smaller number of kernel threads**.
   - Combines advantages of the previous two models.
   - The OS may create **a pool of kernel threads**, and user threads are scheduled dynamically to available kernel threads.
   - **Formula:**

     Number of Kernel Threads ≤ Number of User Threads
   - **Example:**

     Solaris and Windows with Thread Pools use this model.

2. **Multi-Level Queue (MLQ)**
   - In this scheduling method, **processes/jobs are divided into multiple groups** based on their characteristics.
   - Each group is assigned to a **separate queue**.
   - Example of classification:
     - **System processes**
     - **Interactive processes**
     - **Batch processes**
     - **Student or background processes**

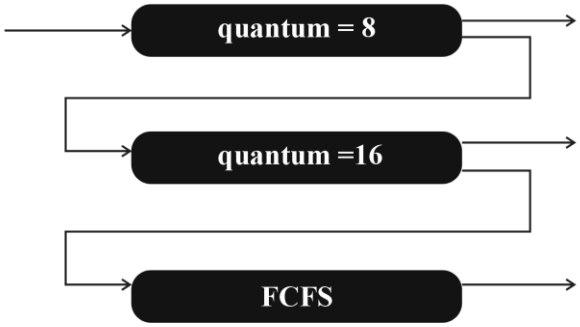
   Key Point:
   - Each queue has its **own scheduling algorithm**.
   - Queues may have **fixed priorities** — higher-priority queues get CPU first.
   - **Example:**
     - System queue (highest priority) uses Round Robin.
     - The batch queue (lower priority) uses FCFS.



3. **Multi-Level Feedback Queue (MLFQ)**
   - In **Multi-Level Queue Scheduling**, once a process is assigned to a queue, it **stays permanently** in that queue.
   - **MLFQ** improves this by allowing **processes to move between queues** depending on their behavior and requirements.

   Advantages
   - Reduces starvation by **dynamically adjusting process priorities**.

   Disadvantages
   - **Higher scheduling overhead** due to constant movement between queues.



Parameters of Multi-Level Feedback Queue

1. **Number of Queues:**
   - Determines how finely processes can be categorized.
2. **Scheduling Algorithm for Each Queue:**
   - **Example:** RR for high-priority, FCFS for low-priority.
3. **Upgrade Method:**
   - Defines when a process should be **moved to a higher-priority queue** (e.g., if it waits too long).
4. **Demotion Method:**
   - Defines when a process should be **moved to a lower-priority queue** (e.g., if it uses too much CPU time).
5. **Queue Entry Criteria:**
   - Determines **which queue** a new process will enter based on its **type or priority**.

Summary of Multithreading

Concept	Explanation / Key Points
Thread	Basic CPU utilization unit containing PC, ID, stack, and registers; shares code/data with other threads.
Motivation	Enables parallelism, faster execution, responsiveness, and resource sharing.
Benefits	Responsiveness, Resource Sharing, Economy, and Scalability.
Multicore Programming	Uses multiple CPU cores efficiently; challenges: task division, load balance, data dependency, and testing.
Models	Many-to-One, One-to-One, and Many-to-Many thread mappings between user and kernel levels.
MLQ Scheduling	Processes are divided into static queues with fixed priorities.
MLFQ Scheduling	Processes can move between queues based on performance; reduces starvation.
MLFQ Parameters	Number of queues, algorithms, upgrade/demotion methods, and entry rules.

मल्टीथ्रेडिंग

कवर किए जाने वाले विषय

1. थ्रेड्स
2. मल्टी-लेवल क्यू
3. मल्टी-लेवल फीडबैक क्यू

1. थ्रेड्स

- CPU उपयोग की मूल इकाई को थ्रेड कहा जाता है।
- एक थ्रेड में निम्नलिखित शामिल होते हैं:
  - **प्रोग्राम काउंटर (PC):** यह अगले निष्पादित होने वाले निर्देश को इंगित करता है।.
  - **थ्रेड ID:** अद्वितीय पहचानकर्ता.
  - **स्टैक:** यह स्थानीय वेरिएबल्स, रिटर्न एड्रेस और फंक्शन कॉल्स को स्टोर करता है।
  - **रजिस्टर सेट:** यह प्रोसेसर रजिस्टर के मानों को स्टोर करता है।
- थ्रेड्स उसी प्रोसेस से संबंधित अन्य थ्रेड्स के साथ कुछ विशिष्ट संसाधनों को साझा करते हैं, जैसे:
  - **डेटा सेक्शन:** इसमें ग्लोबल वेरिएबल्स और स्टेटिक डेटा स्टोर होता है जो सभी थ्रेड्स के लिए सुलभ होता है।
  - **कोड सेक्शन:** इसमें निष्पादन योग्य निर्देश होते हैं जिन्हें सभी थ्रेड्स साझा रूप से रन करते हैं।
  - **ऑपरेटिंग सिस्टम संसाधन:** इसमें ओपन फाइल्स, सिग्नल और अन्य सिस्टम-स्तरीय रिसोर्स शामिल होते हैं।

पारंपरिक (हैवीवेट) प्रोसेस:

- इसमें नियंत्रण का केवल एक थ्रेड होता है।
- यदि वह थ्रेड ब्लॉक हो जाता है (उदाहरण के लिए, I/O का इंतज़ार करते समय), तो पूरा प्रोसेस ब्लॉक हो जाता है।

मल्टीथ्रेडिंग के लिए अभिप्रेरणा:

- अधिकांश आधुनिक एप्लीकेशन मल्टी-थ्रेडेड प्रोग्राम के रूप में कार्यान्वित किए जाते हैं।
- वे एक ही प्रोसेस के रूप में चलते हैं जिसमें नियंत्रण के कई थ्रेड्स शामिल होते हैं।
- यह डेस्कटॉप एप्लीकेशन, वेब सर्वर और आधुनिक ऑपरेटिंग सिस्टम डिजाइन में सामान्य है।

मल्टीथ्रेडेड प्रोग्रामिंग के लाभ

मल्टीथ्रेडिंग के लाभों को चार प्रमुख श्रेणियों में विभाजित किया जा सकता है:

1. **रिस्पॉन्सिवनेस:**
   - यह प्रोग्राम के कुछ हिस्सों को चलते रहने की अनुमति देता है, भले ही अन्य हिस्से अवरुद्ध हों।
   - उदाहरण: एक वेब ब्राउज़र में, एक थ्रेड टेक्स्ट रेंडर कर सकता है जबकि दूसरा थ्रेड इमेज लोड करता है।
2. **संसाधन साझाकरण:**
   - थ्रेड्स कोड और डेटा साझा करते हैं, जिससे उनके बीच संचार आसान और कुशल हो जाता है।
3. **अर्थव्यवस्था:**
   - थ्रेड बनाना और प्रबंधित करना पूर्ण प्रक्रियाओं को बनाने की तुलना में तेज़ और सस्ता है।
4. **स्केलेबिलिटी:**
   - मल्टीकोर सिस्टम पर, थ्रेड्स विभिन्न कोर्स) पर समानांतर रूपसे चल सकते हैं, जिससे प्रदर्शन में सुधार होता है।

मल्टीकोर प्रोग्रामिंग

- एक मल्टीकोर सिस्टम में एक ही चिप पर कई CPU कोर्स होते हैं।
- प्रत्येक कोर ऑपरेटिंग सिस्टम को एक अलग प्रोसेसर के रूप में दिखाई देता है।
- मल्टीथ्रेडेड प्रोग्रामिंग कई कोर्स का प्रभावी ढंग से उपयोग करने में मदद करती है, जिससे कन्करेंसी और थ्रूपुट में सुधार होता है।

मल्टीकोर प्रोग्रामिंग में चुनौतियाँ

1. **गतिविधि का विभाजन:**
   - प्रोग्राम के उन हिस्सों की पहचान करना जिन्हें स्वतंत्र कार्यों में विभाजित किया जा सकता है ताकि उन्हें समांतर रूप से चलाया जा सके।
2. **संतुलन:**
   - यह सुनिश्चित करना कि सभी कोर्स के पास समान मात्रा में कार्य हो ताकि कोई भी कोर खाली न रहे।
3. **डेटा विभाजन:**
   - डेटा को इस तरह अलग करना कि प्रत्येक थ्रेड/कोर अपने आवश्यक डेटा तक स्वतंत्र रूप से पहुँच सके और उसमें बदलाव कर सके।
4. **डेटा निर्भरता:**
   - कार्यों के बीच डेटा निर्भरता (साझा डेटा) की पहचान करना ताकि रेस कंडीशंस और सिंक्रोनाइज़ेशन की समस्याओं से बचा जा सके।
5. **टेस्टिंग और डिबगिंग:**
   - समांतर प्रोग्राम्स की टेस्टिंग करना कठिन है क्योंकि इसमें कई निष्पादन पथ होते हैं।
   - मल्टीथ्रेडेड एप्लिकेशन्स की डिबगिंग करना सिंगल-थ्रेडेड एप्लिकेशन्स की तुलना में अधिक कठिन होता है।

मल्टीथ्रेडिंग मॉडल

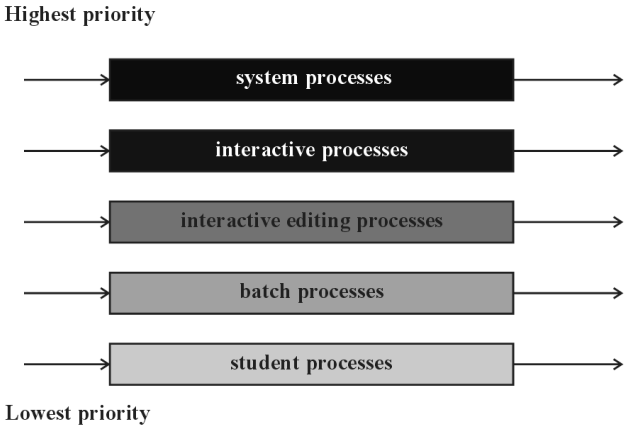
ऑपरेटिंग सिस्टम यूजर-लेवल थ्रेड्स को कर्नल-लेवल थ्रेड्स के साथ मैप करने के लिए विभिन्न मॉडलों का उपयोग करते हैं।

1. **मेनी-टू-वन मॉडल**
   - कई यूजर थ्रेड्स को एक ही कर्नल थ्रेड पर मैप किया जाता है।
   - थ्रेड प्रबंधन एक थ्रेड लाइब्रेरी का उपयोग करके यूजर स्पेस में किया जाता है, जो इसे कुशल बनाता है।
   - हालांकि, यदि एक थ्रेड ब्लॉकिंग सिस्टम कॉल करता है, तो पूरा प्रोसेस ब्लॉक हो जाता है।
   - उदाहरण: जावा ग्रीन थ्रेड्स।
2. **वन-टू-वन मॉडल**
   - प्रत्येक यूजर-लेवल थ्रेड को एक अलग कर्नल थ्रेड के साथ मैप किया जाता है।
   - यह वास्तविक कन्करेंसी की अनुमति देता है — मल्टीप्रोसेसर सिस्टम पर कई थ्रेड्स समांतर रूप से चल सकते हैं।
   - कमी: एक यूजर थ्रेड बनाने के लिए एक कर्नल थ्रेड बनाने की आवश्यकता होती है, जिससे सिस्टम पर ओवरहेड बढ़ जाता है।
   - **उदाहरण:**
     विंडोज और लिनक्स सिस्टम इसी मॉडल (वन-टू-वन मॉडल) का उपयोग करते हैं।
3. **मेनी-टू-मेनी मॉडल**
   - यह कई यूजर थ्रेड्स को कर्नल थ्रेड्स की समान या उससे कम संख्या के साथ मैप करता है।
   - यह पिछले दो मॉडलों के लाभों को संयोजित करता है।
   - ऑपरेटिंग सिस्टम कर्नल थ्रेड्स का एक पूल बना सकता है, और यूजर थ्रेड्स को उपलब्ध कर्नल थ्रेड्स पर गतिशील रूप से शेड्यूल किया जाता है।
   - **सूत्र:** कर्नल थ्रेड्स की संख्या ≤ यूजर थ्रेड्स की संख्या
   - **उदाहरण:** थ्रेड पूल के साथ सोलारिस और विंडोज इसी मॉडल का उपयोग करते हैं।

डिजिटल फंडामेंटल्स और ऑपरेटिंग सिस्टम (OS) शेड्यूलिंग की शब्दावली का उपयोग करते हुए हिंदी अनुवाद नीचे दिया गया है:

2. **मल्टी-लेवल क्यू (Multi-Level Queue - MLQ)**
   - इस शेड्यूलिंग पद्धति में, प्रोसेस/जॉब्स (processes/jobs) को उनकी विशेषताओं के आधार पर कई समूहों में विभाजित किया जाता है।
   - प्रत्येक समूह को एक अलग क्यू (queue) में रखा जाता है।
   - वर्गीकरण के उदाहरण:
     - सिस्टम प्रोसेस
     - इंटरैक्टिव प्रोसेस
     - बैच प्रोसेस
     - स्टूडेंट या बैकग्राउंड प्रोसेस

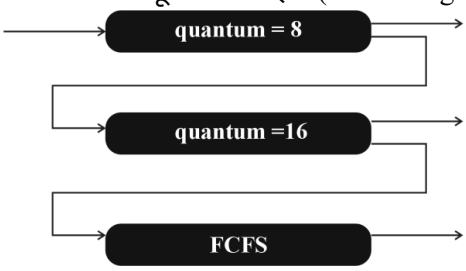
   मुख्य बिंदु:
   - प्रत्येक क्यू का अपना शेड्यूलिंग एल्गोरिथम होता है।
   - क्यू की प्राथमिकताएं निश्चित हो सकती हैं — उच्च-प्राथमिकता वाली क्यू को CPU पहले मिलता है।
   - **उदाहरण:**
     - सिस्टम क्यू (उच्चतम प्राथमिकता) राउंड रॉबिन का उपयोग करती है।
     - बैच क्यू (निम्न प्राथमिकता) FCFS का उपयोग करती है।



3. **मल्टी-लेवल फीडबैक क्यू**
   - मल्टी-लेवल क्यू शेड्यूलिंग में, एक बार जब किसी प्रोसेस को एक क्यू आवंटित कर दी जाती है, तो वह स्थायी रूप से उसी क्यू में रहता है।
   - MLFQ इसमें सुधार करता है और प्रोसेस को उनके व्यवहार और आवश्यकताओं के आधार पर क्यू के बीच स्थानांतरित होने की अनुमति देता है।

   लाभ:
   - प्रोसेस की प्राथमिकताओं को गतिशील रूप से समायोजित करके स्टार्वेशन को कम करता है।

   हानि:
   - क्यू के बीच निरंतर स्थानांतरण के कारण शेड्यूलिंग ओवरहेड (scheduling overhead) अधिक होता है।



मल्टी-लेवल फीडबैक क्यू (MLFQ) के पैरामीटर्स

1. **क्यू की संख्या:**
   - यह निर्धारित करता है कि प्रोसेस को कितनी सूक्ष्मता से वर्गीकृत किया जा सकता है।
2. **प्रत्येक क्यू के लिए शेड्यूलिंग एल्गोरिथम:**
   - **उदाहरण:** उच्च-प्राथमिकता के लिए RR (राउंड रॉबिन) और निम्न-प्राथमिकता के लिए FCFS।
3. **अपग्रेड विधि:**
   - यह परिभाषित करता है कि एक प्रोसेस को उच्च-प्राथमिकता वाली क्यू में कब स्थानांतरित किया जाना चाहिए (उदाहरण के लिए, यदि वह बहुत लंबे समय तक प्रतीक्षा करता है)।
4. **डिमोशन विधि:**
   - यह परिभाषित करता है कि एक प्रोसेस को कम-प्राथमिकता वाली क्यू में कब स्थानांतरित किया जाना चाहिए (उदाहरण के लिए, यदि वह बहुत अधिक CPU समय का उपयोग करता है)।
5. **क्यू एंट्री मानदंड:**
   - यह निर्धारित करता है कि एक नया प्रोसेस अपने प्रकार या प्राथमिकता के आधार पर किस क्यू में प्रवेश करेगा।

मल्टीथ्रेडिंग का सारांश

अवधारणा	स्पष्टीकरण / मुख्य बिंदु
थ्रेड	उपयोग की मूल इकाई जिसमें प्रोग्राम काउंटर, पहचानकर्ता, स्टैक और रजिस्टर्स शामिल होते हैं; यह अन्य थ्रेड्स के साथ कोड/डेटा साझा करता है।
अभिप्रेरणा	यह समांतरता, तीव्र निष्पादन, रिस्पॉन्सिवनेस और संसाधन साझाकरण को सक्षम बनाता है।
लाभ	रिस्पॉन्सिवनेस, संसाधन साझाकरण, अर्थव्यवस्था, और स्केलेबिलिटी।
मल्टीकोर प्रोग्रामिंग	कई कोर्स का कुशलतापूर्वक उपयोग करता है; चुनौतियाँ: कार्य विभाजन, लोड बैलेंस, डेटा डिपेंडेंसी और टेस्टिंग।
मॉडल	यूजर और कर्नल स्तरों के बीच मेनी-टू-वन, वन-टू-वन, और मेनी-टू-मेनी थ्रेड मैपिंग।
शेड्यूलिंग	प्रोसेस को निश्चित प्राथमिकताओं के साथ स्थिर क्यू में विभाजित किया जाता है।
शेड्यूलिंग	प्रोसेस प्रदर्शन के आधार पर क्यू के बीच जा सकते हैं; यह 'स्टार्वेशन' को कम करता है।
पैरामीटर्स	क्यू की संख्या, एल्गोरिदम, अपग्रेड/डिमोट करने के तरीके और एंट्री रूल्स।



PW Web/App: https://smart.link/7wwosivoicgd4
Library- https://smart.link/sdfez8ejd80if